UNIVERSIDADE PRESBITERIANA MACKENZIE
FACULDADE DE CIÊNCIA DA COMPUTAÇÃO

Filipe Soares – RA 10734981
Miguel Soares – RA 10734935
Julio Freire – RA 10734961

PROJETO EXTENSIONISTA: SISTEMA INTEGRADO PARA A INDÚSTRIA FARMACÊUTICA

Parte 3 – Arquitetura MVC, Diagrama de Classes e Implementação de Recorte Funcional em Python

São Paulo
2026

SUMÁRIO

1 INTRODUÇÃO 3

2 ARQUITETURA DE SOFTWARE: PADRÃO MVC 3

2.1 Conceito e Responsabilidades no Padrão MVC 3

2.2 Fluxo de Execução e Justificativa da Escolha 4

3 ESPECIFICAÇÃO DO RECORTE FUNCIONAL 4

4 DIAGRAMA DE CLASSES ADAPTADO AO PADRÃO MVC 5

4.1 Decisões de Modelagem e Distribuição nas Camadas MVC 5

4.2 Especificação PlantUML do Diagrama de Classes MVC 7

5 IMPLEMENTAÇÃO EM PYTHON (ARQUITETURA MVC) 9

5.1 Código-Fonte da Camada Model 9

5.2 Código-Fonte das Camadas View, Controller e Testes 11

5.3 Saída da Execução no Console 13

REFERÊNCIAS 15

1 INTRODUÇÃO

Este documento apresenta a definição arquitetural e a implementação prática de um recorte funcional do Sistema Integrado para a Indústria Farmacêutica, dando continuidade à especificação de requisitos e à modelagem elaborada na *Unified Modeling Language* — UML (OBJECT MANAGEMENT GROUP, 2017). O objetivo desta etapa consiste em aplicar o padrão arquitetural *Model-View-Controller* (MVC) sobre um caso de uso do domínio, detalhando seu respectivo diagrama de classes e sua codificação em linguagem Python.

Para atender ao escopo proposto, selecionou-se o fluxo de controle de qualidade sobre lotes de matéria-prima recebidos em quarentena, correspondente ao requisito funcional RF-QUA-01 e ao caso de uso UC03 (Avaliar lote de material). Conforme as convenções estabelecidas no projeto, todos os artefatos de código e diagramação preservam rigorosamente os mesmos identificadores de classes, atributos, métodos e estados em letras maiúsculas, tais como QUARENTENA, APROVADO e REPROVADO, em alinhamento às diretrizes de Boas Práticas de Fabricação (BRASIL, 2022) e de proteção de dados (BRASIL, 2018).

2 ARQUITETURA DE SOFTWARE: PADRÃO MVC

A arquitetura de software compreende o conjunto de decisões estruturais relativas aos componentes de um sistema, às formas de comunicação entre eles e aos princípios que orientam sua evolução. Tais decisões determinam atributos essenciais de qualidade, incluindo manutenibilidade, escalabilidade, desempenho, testabilidade e resiliência. Por se tratarem de definições de difícil reversão, a escolha arquitetural deve considerar o domínio do problema, o tamanho da equipe e o momento evolutivo do produto.

2.1 Conceito e Responsabilidades no Padrão MVC

Proposto por Trygve Reenskaug no final da década de 1970 no ambiente Smalltalk, o padrão MVC organiza internamente a aplicação por meio da separação de responsabilidades em três camadas distintas:

a) **Model (Modelo):** concentra os dados, o estado das entidades e as regras de negócio da aplicação, sem possuir qualquer conhecimento sobre como essas informações serão exibidas ou formatadas;
b) **View (Visão):** encarrega-se exclusivamente da apresentação dos dados ao usuário, podendo ser materializada por telas, páginas HTML ou respostas estruturadas em texto e JSON;

c) **Controller (Controlador):** recebe a entrada ou requisição externa, aciona o Model para processar as regras de negócio ou consultar a base de dados e seleciona a View responsável por renderizar a resposta.

2.2 Fluxo de Execução e Justificativa da Escolha

O funcionamento típico da arquitetura MVC desenvolve-se em cinco etapas sequenciais: (1) o usuário envia uma requisição ou comando; (2) o Controller recebe a solicitação e invoca o Model; (3) o Model recupera ou atualiza os dados aplicando as validações de negócio e devolve o resultado ao Controller; (4) o Controller entrega os dados processados à View; e (5) a View formata e devolve a resposta final ao usuário.

O Quadro 1 sintetiza as características arquiteturais do padrão MVC que fundamentam sua adoção para o recorte do Sistema Integrado para a Indústria Farmacêutica.

Quadro 1 – Síntese das características do padrão arquitetural MVC

Critério Arquitetural	Caracterização no Padrão MVC
Foco Principal	Separar apresentação (View), dados e regras de negócio (Model) e controle de fluxo (Controller).
Nível de Aplicação	Organização interna da aplicação, ideal para sistemas administrativos e módulos de informação de complexidade baixa a média.
Acoplamento e Complexidade	Acoplamento moderado e complexidade baixa, permitindo reutilizar a mesma regra de negócio do Model em diferentes formatos de saída na View.
Vantagens e Limitações	Oferece clara divisão de responsabilidades e evolução paralela, exigindo cuidado para evitar controladores sobrecarregados (*fat controllers*) em sistemas extensos.

Fonte: Elaborado pelos autores com base no material de Arquitetura de Sistemas.

3 ESPECIFICAÇÃO DO RECORTE FUNCIONAL

O recorte funcional selecionado abrange a decisão de qualidade sobre lotes de matéria-prima recém-recebidos pelo almoxarifado. Ao ingressar no estoque, todo lote de material recebe o status inicial QUARENTENA, ficando bloqueado para consumo em ordens de produção até que um Analista da Qualidade registre formalmente sua aprovação ou reprovação.

Quadro 2 – Requisitos e casos de uso contemplados no recorte funcional

Código	Classificação	Descrição Especificada no Projeto
RF-QUA-01	Requisito Funcional Principal	O sistema deve permitir que o Analista da Qualidade aprove ou reprove um lote de material em QUARENTENA, registrando a justificativa.
RF-QUA-03	Requisito Funcional Complementar	O sistema deve manter o histórico das decisões de qualidade de cada lote, com usuário, data e hora, decisão e justificativa.
RNF-03	Requisito Não Funcional (Acesso)	Cada ação deve ser permitida somente aos perfis autorizados (restrito ao perfil QUALIDADE).
RNF-06	Requisito Não Funcional (Confiabilidade)	O sistema deve rejeitar entradas inválidas ou campos obrigatórios vazios com mensagem clara, sem interromper a execução.
UC03	Caso de Uso Base	Avaliar lote de material: aprova ou reprova o lote em quarentena (Ator principal: Analista da Qualidade).
UC14	Caso de Uso de Extensão (*extend*)	Registrar reprovação de lote: registra a reprovação com justificativa obrigatória quando a decisão da qualidade for negativa em UC03.

Fonte: Elaborado pelos autores.

4 DIAGRAMA DE CLASSES ADAPTADO AO PADRÃO MVC

Na modelagem UML (OBJECT MANAGEMENT GROUP, 2017), o recorte de avaliação de qualidade articula as classes da vista de materiais, qualidade e estoque com a classe de controle de acesso. Sob a arquitetura MVC, essas classes são distribuídas nas três camadas estruturais, garantindo que as regras de estado permaneçam encapsuladas no Model.

4.1 Decisões de Modelagem e Distribuição nas Camadas MVC

A adaptação do diagrama de classes para o padrão MVC segue quatro decisões estruturais diretamente alinhadas à especificação original:

a) **Encapsulamento no Model:** a classe `LoteMaterial` concentra o atributo `status` (do tipo enumerado `StatusLoteMaterial`) e os métodos de negócio `aprovar()`, `reprovar()`, `estaVencido()`, `podeSerConsumido()` e `baixarSaldo()`. O lote nasce em `QUARENTENA` e somente transita para `APROVADO` ou `REPROVADO` mediante validação de estado e de permissão;

b) **Rastreabilidade de Decisões e Perfis no Model:** cada avaliação gera uma instância de `DecisaoQualidade` vinculada ao `LoteMaterial` e ao `Usuario` que a registrou, armazenando a enumeração `Decisao` (`APROVADO` ou `REPROVADO`), a justificativa e o carimbo de data e hora;

c) **Isolamento da Apresentação na View:** a classe `AvaliacaoQualidadeView` não altera estados nem acessa o banco de dados, limitando-se a receber

os objetos de domínio e formatar respostas estruturadas em JSON para exibição no console;

d) **Orquestração no Controller:** a classe `AvaliacaoQualidadeController` expõe os métodos acionados pelo usuário (`avaliarLoteMaterial` e `consultarHistoricoLote`), recupera as entidades de domínio, invoca os métodos do Model e repassa o resultado para a View.

Quadro 3 – Especificação das classes do recorte funcional por camada MVC

Camada MVC	Classe / Enumeração	Atributos Principais	Operações (Métodos)
Model	`LoteMaterial`	`id: Long, numeroLote: String, loteFornecedor: String, quantidadeRecebida: Decimal, saldoAtual: Decimal, dataRecebimento: Date, dataValidade: Date, status: StatusLoteMaterial`	`+aprovar(): void +reprovar(): void +estaVencido(): boolean +podeSerConsumido(qtd: Decimal): boolean +baixarSaldo(qtd: Decimal): void`
Model	`DecisaoQualidade`	`id: Long, decisao: Decisao, justificativa: String, dataHora: DateTime`	Construtor de registro imutável vinculado a `Usuario` e `LoteMaterial`.
Model	`MateriaPrima`	`id: Long, codigo: String, nome: String, unidadeMedida: String, estoqueMinimo: Decimal`	Entidade cadastral associada a `LoteMaterial` (1 possui 0..*).
Model	`Fornecedor`	`id: Long, razaoSocial: String, cnpj: String, qualificado: boolean`	Entidade cadastral associada a `LoteMaterial` (1 fornece 0..*).
Model	`Usuario`	`id: Long, nome: String, login: String, senhaHash: String, perfil: PerfilUsuario, ativo: boolean`	`+autenticar(senha: String): boolean +temPermissao(acao: String): boolean`
Model	`StatusLoteMaterial, Decisao, PerfilUsuario`	Valores enumerados restritos às regras de negócio do domínio farmacêutico.	Restrição estática de estados e perfis.
View	`AvaliacaoQualidadeView`	Sem estado de negócio.	`+renderizarSucesso(lote, decisao): String +renderizarHistorico (lote): String +renderizarErro(mensagem) : String`
Controller	`AvaliacaoQualidadeController`	`banco_lotes: Map<Long, LoteMaterial>, view: AvaliacaoQualidadeView`	`+avaliarLote(lote_id, usuario, decisao, justificativa): String +consultarHistoricoLote (lote_id): String`

Fonte: Elaborado pelos autores.

4.2 Especificação PlantUML do Diagrama de Classes MVC

Em conformidade com o padrão de documentação adotado no projeto, apresenta-se a seguir a descrição formal em PlantUML do Diagrama de Classes do recorte funcional estruturado nas camadas Model, View e Controller:

```
@startuml
skinparam classAttributeIconSize 0

package "Camada Controller" {
    class AvaliacaoQualidadeController {
        - banco_lotes : Map<Long, LoteMaterial>
        - view : AvaliacaoQualidadeView
        + avaliarLote(lote_id : Long, usuario : Usuario, decisao : String,
            justificativa : String) : String
        + consultarHistoricoLote(lote_id : Long) : String
    }
}

package "Camada View" {
    class AvaliacaoQualidadeView {
        + renderizarSucesso(lote : LoteMaterial, decisao : DecisaoQualidade) : String
        + renderizarHistorico(lote : LoteMaterial) : String
        + renderizarErro(mensagem : String) : String
    }
}

package "Camada Model" {
    class Fornecedor {
        - id : Long
        - razao_social : String
        - cnpj : String
        - qualificado : boolean
    }

    class MateriaPrima {
        - id : Long
        - codigo : String
        - nome : String
        - unidade_medida : String
        - estoque_minimo : Decimal
    }

    class LoteMaterial {
        - id : Long
        - numeroLote : String
        - loteFornecedor : String
        - quantidadeRecebida : Decimal
        - saldoAtual : Decimal
        - dataRecebimento : Date
        - dataValidade : Date
        - status : String
        + aprovar(usuario : Usuario, justificativa : String) : DecisaoQualidade
        + reprovar(usuario : Usuario, justificativa : String) : DecisaoQualidade
        + estaVencido() : boolean
        + podeSerConsumido(qtd : Decimal) : boolean
        + baixarSaldo(qtd : Decimal) : void
    }

    class DecisaoQualidade {
        - id : Long
        - decisao : String
        - justificativa : String
```

```
        - dataHora : DateTime
    }

    class Usuario {
        - id : Long
        - nome : String
        - login : String
        - senhaHash : String
        - perfil : String
        - ativo : boolean
        + autenticar(senha : String) : boolean
        + temPermissao(acao : String) : boolean
    }
}

Fornecedor "1" -- "0..*" LoteMaterial : fornece
MateriaPrima "1" -- "0..*" LoteMaterial : possui
LoteMaterial "1" -- "0..*" DecisaoQualidade : tem
Usuario "1" -- "0..*" DecisaoQualidade : registra
AvaliacaoQualidadeController --> AvaliacaoQualidadeView : usa
AvaliacaoQualidadeController --> LoteMaterial : avalia

@enduml
```

5 IMPLEMENTAÇÃO EM PYTHON (ARQUITETURA MVC)

A codificação do recorte funcional foi desenvolvida no arquivo `projeto_mvc.py` utilizando orientação a objetos direta em Python e apenas os módulos padrão `json` e `datetime`, sem dependência de interface gráfica. A camada View renderiza as saídas em formato JSON estruturado, demonstrando o desacoplamento entre as regras de negócio do domínio farmacêutico e a apresentação dos resultados.

5.1 Código-Fonte da Camada Model

O trecho a seguir implementa as classes de domínio da camada Model (`Fornecedor`, `MateriaPrima`, `Usuario`, `DecisaoQualidade` e `LoteMaterial`), preservando os atributos e métodos definidos no Diagrama de Classes:

```
import json
from datetime import date, datetime

# ========================================
# MODEL - Classes de Dominio e Regras
# ========================================

class Fornecedor:
    def __init__(self, id, razao_social, cnpj, qualificado=True):
        self.id = id
        self.razao_social = razao_social
        self.cnpj = cnpj
        self.qualificado = qualificado


class MateriaPrima:
    def __init__(self, id, codigo, nome, unidade_medida, estoque_minimo):
        self.id = id
        self.codigo = codigo
        self.nome = nome
        self.unidade_medida = unidade_medida
        self.estoque_minimo = estoque_minimo


class Usuario:
    def __init__(self, id, nome, login, senha, perfil, ativo=True):
        self.id = id
        self.nome = nome
        self.login = login
        self.senhaHash = "hash_" + senha
        self.perfil = perfil
        self.ativo = ativo

    def autenticar(self, senha):
        return self.ativo and self.senhaHash == ("hash_" + senha)

    def temPermissao(self, acao):
        if not self.ativo:
            return False
        if acao == "AVALIAR_LOTE_MATERIAL" and self.perfil == "QUALIDADE":
            return True
        return False
```

```
class DecisaoQualidade:
    def __init__(self, id, decisao, justificativa, usuario):
        self.id = id
        self.decisao = decisao
        self.justificativa = justificativa
        self.usuario = usuario
        self.dataHora = datetime.now().strftime("%Y-%m-%d %H:%M:%S")


class LoteMaterial:
    def __init__(self, id, numero_lote, lote_fornecedor, materia_prima, fornecedor,
        qtd_recebida, data_recebimento, data_validade):
        self.id = id
        self.numeroLote = numero_lote
        self.loteFornecedor = lote_fornecedor
        self.materiaPrima = materia_prima
        self.fornecedor = fornecedor
        self.quantidadeRecebida = qtd_recebida
        self.saldoAtual = qtd_recebida
        self.dataRecebimento = data_recebimento
        self.dataValidade = data_validade
        self.status = "QUARENTENA"
        self.historicoDecisoes = []

    def estaVencido(self):
        return self.dataValidade < date.today()

    def podeSerConsumido(self, qtd):
        return self.status == "APROVADO" and not self.estaVencido() and
            self.saldoAtual >= qtd and qtd > 0

    def baixarSaldo(self, qtd):
        if not self.podeSerConsumido(qtd):
            raise Exception("Lote nao pode ser consumido ou saldo insuficiente.")
        self.saldoAtual -= qtd
        if self.saldoAtual == 0:
            self.status = "ESGOTADO"

    def aprovar(self, usuario, justificativa):
        if not usuario.temPermissao("AVALIAR_LOTE_MATERIAL"):
            raise Exception(f"Acesso negado: usuario '{usuario.nome}'
                ({usuario.perfil}) nao tem perfil QUALIDADE.")
        if self.status != "QUARENTENA":
            raise Exception(f"Bloqueado: o lote '{self.numeroLote}' esta com status
                '{self.status}'. So lotes em QUARENTENA podem ser avaliados.")
        if not justificativa or justificativa.strip() == "":
            raise Exception("A justificativa e obrigatoria.")
        if self.estaVencido():
            self.status = "VENCIDO"
            raise Exception("Bloqueado: o lote esta vencido.")

        self.status = "APROVADO"
        nova_decisao = DecisaoQualidade(len(self.historicoDecisoes) + 1, "APROVADO",
            justificativa, usuario)
        self.historicoDecisoes.append(nova_decisao)
        return nova_decisao

    def reprovar(self, usuario, justificativa):
        if not usuario.temPermissao("AVALIAR_LOTE_MATERIAL"):
            raise Exception(f"Acesso negado: usuario '{usuario.nome}'
                ({usuario.perfil}) nao tem perfil QUALIDADE.")
        if self.status != "QUARENTENA":
            raise Exception(f"Bloqueado: o lote '{self.numeroLote}' esta com status
                '{self.status}'. So lotes em QUARENTENA podem ser avaliados.")
        if not justificativa or justificativa.strip() == "":
            raise Exception("A justificativa e obrigatoria para reprovar o lote.")
```

```
        self.status = "REPROVADO"
        nova_decisao = DecisaoQualidade(len(self.historicoDecisoes) + 1, "REPROVADO",
            justificativa, usuario)
        self.historicoDecisoes.append(nova_decisao)
        return nova_decisao
```

5.2 Código-Fonte das Camadas View, Controller e Testes

O bloco complementar apresenta a implementação da `AvaliacaoQualidadeView`, do `AvaliacaoQualidadeController` e a rotina de execução dos cinco cenários de teste:

```
# ========================================
# VIEW - Exibicao dos Resultados (JSON/Texto)
# ========================================

class AvaliacaoQualidadeView:
    def renderizarSucesso(self, lote, decisao):
        dados = {
            "status_requisicao": "SUCESSO",
            "lote": lote.numeroLote,
            "materia_prima": lote.materiaPrima.nome,
            "novo_status": lote.status,
            "decisao": {
                "id": decisao.id,
                "resultado": decisao.decisao,
                "justificativa": decisao.justificativa,
                "analista": decisao.usuario.nome,
                "data_hora": decisao.dataHora
            }
        }
        return json.dumps(dados, indent=4, ensure_ascii=False)

    def renderizarHistorico(self, lote):
        lista = []
        for d in lote.historicoDecisoes:
            lista.append({
                "id": d.id,
                "decisao": d.decisao,
                "justificativa": d.justificativa,
                "usuario": d.usuario.nome,
                "data_hora": d.dataHora
            })
        dados = {
            "lote": lote.numeroLote,
            "status_atual": lote.status,
            "historico": lista
        }
        return json.dumps(dados, indent=4, ensure_ascii=False)

    def renderizarErro(self, mensagem):
        dados = {
            "status_requisicao": "ERRO",
            "motivo": mensagem
        }
        return json.dumps(dados, indent=4, ensure_ascii=False)


# ========================================
# CONTROLLER - Intermediario entre View e Model
```

```
# ========================================

class AvaliacaoQualidadeController:
    def __init__(self, banco_lotes, view):
        self.banco_lotes = banco_lotes
        self.view = view

    def avaliarLote(self, lote_id, usuario, decisao, justificativa):
        lote = self.banco_lotes.get(lote_id)
        if not lote:
            return self.view.renderizarErro("Lote nao encontrado.")

        try:
            if decisao == "APROVADO":
                reg = lote.aprovar(usuario, justificativa)
            elif decisao == "REPROVADO":
                reg = lote.reprovar(usuario, justificativa)
            else:
                return self.view.renderizarErro("Decisao invalida.")

            return self.view.renderizarSucesso(lote, reg)
        except Exception as erro:
            return self.view.renderizarErro(str(erro))

    def consultarHistoricoLote(self, lote_id):
        lote = self.banco_lotes.get(lote_id)
        if not lote:
            return self.view.renderizarErro("Lote nao encontrado.")
        return self.view.renderizarHistorico(lote)


# ========================================
# TESTES (Simulando o uso do sistema)
# ========================================

if __name__ == "__main__":
    # Criando dados basicos para testar
    fornecedor1 = Fornecedor(1, "BioQuimica Insumos Ltda", "12.345.678/0001-99")
    mp1 = MateriaPrima(10, "MP-001", "Paracetamol Po", "kg", 50.0)

    lote101 = LoteMaterial(101, "LM-2026-001", "FORN-889", mp1, fornecedor1, 200.0,
        date(2026, 11, 1), date(2028, 11, 1))
    lote102 = LoteMaterial(102, "LM-2026-002", "FORN-890", mp1, fornecedor1, 150.0,
        date(2026, 11, 2), date(2028, 11, 2))

    analista = Usuario(1, "Dra. Helena Costa", "helena.qualidade", "123456",
        "QUALIDADE")
    almoxarife = Usuario(2, "Carlos Mendes", "carlos.almox", "654321", "ALMOXARIFE")

    # Dicionario simulando o banco de dados em memoria
    banco_lotes = {
        101: lote101,
        102: lote102
    }

    view = AvaliacaoQualidadeView()
    controller = AvaliacaoQualidadeController(banco_lotes, view)

    print("--- CENARIO 1: Aprovando lote 101 em Quarentena (UC03) ---")
    print(controller.avaliarLote(101, analista, "APROVADO", "Laudo conferido e
        material aprovado."))

    print("\n--- CENARIO 2: Reprovando lote 102 em Quarentena (UC03 / UC14) ---")
    print(controller.avaliarLote(102, analista, "REPROVADO", "Embalagem violada e com
        umidade."))
```

```
    print("\n--- CENARIO 3: Tentativa de aprovacao por Almoxarife (Bloqueio de
        Perfil) ---")
    print(controller.avaliarLote(101, almoxarife, "APROVADO", "Tentando aprovar pelo
        almoxarifado."))

    print("\n--- CENARIO 4: Tentativa de avaliar lote que nao esta mais em Quarentena
        ---")
    print(controller.avaliarLote(101, analista, "REPROVADO", "Tentando mudar lote ja
        aprovado."))

    print("\n--- CENARIO 5: Consultando historico do lote 101 ---")
    print(controller.consultarHistoricoLote(101))
```

5.3 Saída da Execução no Console

A execução do arquivo `projeto_mvc.py` gera a saída a seguir, comprovando o funcionamento das regras de negócio e dos bloqueios de segurança:

```
--- CENARIO 1: Aprovando lote 101 em Quarentena (UC03) ---
{
    "status_requisicao": "SUCESSO",
    "lote": "LM-2026-001",
    "materia_prima": "Paracetamol Po",
    "novo_status": "APROVADO",
    "decisao": {
        "id": 1,
        "resultado": "APROVADO",
        "justificativa": "Laudo conferido e material aprovado.",
        "analista": "Dra. Helena Costa",
        "data_hora": "2026-11-08 12:50:52"
    }
}

--- CENARIO 2: Reprovando lote 102 em Quarentena (UC03 / UC14) ---
{
    "status_requisicao": "SUCESSO",
    "lote": "LM-2026-002",
    "materia_prima": "Paracetamol Po",
    "novo_status": "REPROVADO",
    "decisao": {
        "id": 1,
        "resultado": "REPROVADO",
        "justificativa": "Embalagem violada e com umidade.",
        "analista": "Dra. Helena Costa",
        "data_hora": "2026-11-08 12:50:52"
    }
}

--- CENARIO 3: Tentativa de aprovacao por Almoxarife (Bloqueio de Perfil) ---
{
    "status_requisicao": "ERRO",
    "motivo": "Acesso negado: usuario 'Carlos Mendes' (ALMOXARIFE) nao tem perfil
        QUALIDADE."
}

--- CENARIO 4: Tentativa de avaliar lote que nao esta mais em Quarentena ---
{
    "status_requisicao": "ERRO",
    "motivo": "Bloqueado: o lote 'LM-2026-001' esta com status 'APROVADO'. So lotes
        em QUARENTENA podem ser avaliados."
}
```

```
--- CENARIO 5: Consultando historico do lote 101 ---
{
    "lote": "LM-2026-001",
    "status_atual": "APROVADO",
    "historico": [
        {
            "id": 1,
            "decisao": "APROVADO",
            "justificativa": "Laudo conferido e material aprovado.",
            "usuario": "Dra. Helena Costa",
            "data_hora": "2026-11-08 12:50:52"
        }
    ]
}
```

REFERÊNCIAS

BRASIL. Agência Nacional de Vigilância Sanitária. **Resolução da Diretoria Colegiada RDC nº 658, de 30 de março de 2022**. Dispõe sobre as Diretrizes Gerais de Boas Práticas de Fabricação de Medicamentos. Brasília, DF: ANVISA, 2022.

BRASIL. **Lei nº 13.709, de 14 de agosto de 2018**. Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília, DF: Presidência da República, 2018.

OBJECT MANAGEMENT GROUP. **Unified Modeling Language (UML)**: specification, version 2.5.1. Needham: OMG, 2017.